INF563 · VÉRIFICATION ET VALIDATION DE LOGICIEL · MASTER 2 SIGL

Devoir 2 – Application To-Do (projets, IndexedDB) en JavaScript : BDD, couverture et mutation

Une application web dont les tâches appartiennent à des **projets** et sont sauvegardées dans **IndexedDB**, réécrite **entièrement en JavaScript**. Ses fonctionnalités sont décrites en Gherkin et vérifiées dans le navigateur ; son code métier est mesuré par la **couverture** (Istanbul, l'équivalent de JaCoCo) et par le **test de mutation** (StrykerJS, l'équivalent de PIT), comme au chapitre V.

100 %
score de mutation (StrykerJS) : 185 mutants, 0 survivant ; 89,73 % avant l'analyse des survivants

100 %
couverture des branches du code métier (Istanbul), avec 51 tests unitaires (Vitest)

24 / 24
scénarios BDD réussis (Cucumber-js + Playwright), 200 étapes

Vert
pipeline GitHub Actions : couverture, mutation et BDD en 69 s

Dépôt du devoir

github.com/dradone237/inf563-bdd-todo-indexeddb · branche `main` · 14 commits · la version Java reste consultable avec l'étiquette `v1-java` · instantané des rapports de couverture et de mutation dans `docs/rapports/`.

Enseignant : Evaris Fomekong · Chapitre V « Couverture et test de mutation », avec les séances « Niveaux, doublures et TDD/BDD » et « Bout en bout web : Playwright et page objects »

HTML · CSS · JavaScript (modules ES) · IndexedDB · Node.js 24 · Vitest 5.1 · Istanbul · StrykerJS 10.1 · Cucumber-js 13.3 · Playwright 1.65 · GitHub Actions

Rapport de devoir · version 2 (JavaScript) · 11 novembre 2026

Table des matières

1. Énoncé et nouvelles consignes **3**
- 1.1 Consignes du devoir 3
- 1.2 Consignes complémentaires de l'enseignant 3
- 1.3 Ce que ce rapport démontre 3

2. Du Java au JavaScript **4**
- 2.1 Correspondance des outils 4
- 2.2 Ce qui change dans l'application 4
- 2.3 Ce qui ne change pas : la spécification 5

3. L'application To-Do **5**
- 3.1 Fonctionnalités 5
- 3.2 Modèle de données IndexedDB 5
- 3.3 Organisation du code 5

4. Stratégie de test **6**
- 4.1 Trois niveaux, deux mesures 6
- 4.2 Arborescence 6
- 4.3 Bibliothèques 7

5. Tests unitaires et doublures **7**
- 5.1 Règles métier : des fonctions pures 7
- 5.2 Persistance : une fake et une horloge injectée 8

6. Couverture de code avec Istanbul **8**
- 6.1 Mise en place : l'équivalent de JaCoCo 8
- 6.2 Lire un rapport Istanbul 9
- 6.3 Résultats : T1 puis T2 9
- 6.4 Ce que la couverture ne dit pas 10

7. Test de mutation avec StrykerJS **10**
- 7.1 Principe et mise en place : l'équivalent de PIT 10
- 7.2 Les opérateurs de mutation 11
- 7.3 Suite T1 : 89,73 % 12
- 7.4 Analyse des survivants 12
- 7.5 Suite T2 : 100 % 13
- 7.6 Ce que montre la comparaison T1 / T2 15

8. Défaut détecté par les tests unitaires **15**

9. Scénarios BDD en JavaScript **16**
- 9.1 Cucumber-js et Playwright 16
- 9.2 Deux oracles : l'écran et la base 16
- 9.3 Couverture des scénarios 16

10. Intégration continue : chaque push lance les trois mesures **17**

11. Résultats **18**
- 11.1 Synthèse des mesures 18
- 11.2 Historique Git 18

12. Lacunes de spécification et décisions **19**

13. Limites et pistes d'amélioration **19**

14. Conclusion **19**

Annexe A – Configuration du projet **21**

Annexe B – Règles métier (src/js/regles.js) **22**

Annexe C – Persistance (src/js/db.js) **23**

Annexe D – Tests ajoutés après le test de mutation (suite T2) **25**

Annexe E – Fichiers .feature (inchangés depuis la version 1) **28**

Annexe F – Pipeline .github/workflows/ci.yml **31**

Annexe G – Commandes utiles **32**

1. Énoncé et nouvelles consignes

1.1 Consignes du devoir

- réaliser une application **To-Do** dans laquelle on peut **ajouter** une tâche et **modifier** une tâche ;
- les tâches **appartiennent à des projets** ;
- les données sont **sauvegardées dans IndexedDB**, la base de données intégrée au navigateur ;
- **toutes les fonctionnalités** sont testées en **BDD**, dans le navigateur ;
- un **push** sur le dépôt **déclenche les tests**.

La première version du devoir répondait à ces consignes en Java (Maven, Cucumber-JVM, Playwright pour Java) : 24 scénarios Gherkin, verts en intégration continue. Elle est conservée dans le dépôt sous l'étiquette `v1-java`.

1.2 Consignes complémentaires de l'enseignant

> **Consignes**
>
> - le projet doit être **entièrement et totalement en JavaScript** ;
> - appliquer la **couverture de code avec JaCoCo** et les **tests de mutation PIT** démontrés au chapitre précédent, avec les **bibliothèques et paquets propres à JavaScript**.

JaCoCo et PIT sont des outils Java : ils ne s'appliquent pas à du JavaScript. La consigne demande donc d'appliquer la même **méthode** avec leurs équivalents JavaScript : **Istanbul** pour la couverture et **StrykerJS** pour la mutation. Le TP du chapitre V (« Renforcer la suite ») fixe cette méthode :

1. implémenter le code et ses tests unitaires ;
2. mesurer la couverture et viser 100 % des branches ;
3. lancer le test de mutation, relever le score et lister les survivants ;
4. pour chaque survivant : ajouter le test qui le tue, ou justifier par écrit qu'il est équivalent ;
5. objectif : score de mutation d'au moins 85 % hors équivalents justifiés, avec les seuils de couverture et de mutation écrits dans la configuration du build ;
6. livrable : code, tests, rapports de couverture et de mutation, tableau des survivants analysés.

1.3 Ce que ce rapport démontre

- le projet ne contient plus une ligne de Java : application, tests, scénarios et outillage sont en JavaScript (section 2) ;
- les **24 scénarios Gherkin sont inchangés** et toujours verts : la migration n'a pas changé le comportement (section 2.3) ;
- la couverture est mesurée par Istanbul avec un seuil qui fait échouer le build (section 6) ;
- le test de mutation est appliqué par StrykerJS avec un seuil de 85 % ; les **19 mutants** non tués par la première suite sont **analysés un par un**, puis tués par une seconde suite (section 7) ;
- les tests unitaires ont trouvé un **défaut réel** que les scénarios BDD ne pouvaient pas voir (section 8) ;
- chaque push lance les trois mesures dans GitHub Actions (section 10).

2. Du Java au JavaScript

2.1 Correspondance des outils

Rôle	Version 1 (Java)	Version 2 (JavaScript)
Build et dépendances	Maven, `pom.xml`	npm, `package.json` et `package-lock.json` (versions exactes figées)
Tests unitaires	JUnit 5	**Vitest**
Couverture de code	**JaCoCo** : `prepare-agent` instrumente le bytecode, `report`, `check`	**Istanbul** : instrumente le JavaScript, produit le rapport et applique les seuils (via Vitest pour les tests unitaires, via `istanbul-lib-instrument` et **nyc** pour les scénarios)
Seuil de couverture	`<counter>BRANCH</counter>` … `<minimum>0.80</minimum>`	`thresholds` dans `vitest.config.js` ; `"branches": 80` dans `.nycrc.json`
Test de mutation	**PIT** : `pitest-maven`, but `mutationCoverage`	**StrykerJS** : `@stryker-mutator/core` et `@stryker-mutator/vitest-runner`
Code ciblé	`<targetClasses>`	`mutate: ['src/js/regles.js', 'src/js/db.js']`
Seuil de mutation	`<mutationThreshold>85</mutationThreshold>`	`thresholds: { break: 85 }`
BDD	Cucumber-JVM	**Cucumber-js**, avec les mêmes fichiers `.feature` en français
Navigateur	Playwright pour Java	**Playwright** (`@playwright/test`)
Serveur HTTP des tests	`SimpleFileServer` du JDK	module `node:http` de Node.js (`outils/serveur.js`)
Doublure d'IndexedDB	aucune	**fake-indexeddb** : une *fake*, vraie implémentation d'IndexedDB en mémoire
Intégration continue	`mvn -B verify`	`npm ci`, puis un script npm par mesure

2.2 Ce qui change dans l'application

En version 1, toute la logique vivait dans l'interface (`app.js`) : une règle comme « un nom fait d'espaces est refusé » ne pouvait être vérifiée qu'en pilotant un navigateur. Or la couverture et la mutation s'appliquent à des tests **unitaires** : il faut d'abord **rendre le code mesurable**. Trois changements, sans modifier le comportement :

- `src/js/regles.js` (nouveau) regroupe les règles métier en **fonctions pures**, sans DOM ni IndexedDB : valider un nom de projet ou un titre, refuser les doublons, lire un identifiant, filtrer les tâches, accorder le compteur (« 1 tâche », « 2 tâches »), choisir le projet par défaut ;
- `src/js/db.js` devient un module ES dont la fonction `ouvrirBase()` reçoit la **fabrique IndexedDB** et l'**horloge** en paramètres. Le navigateur utilise les vraies ; les tests branchent une base en mémoire et une horloge fixe. Une méthode `fermer()` s'ajoute ;
- `src/js/app.js` ne fait plus qu'afficher et relayer les actions : il appelle `regles.js` pour décider et `db.js` pour lire et écrire. `index.html` le charge comme module (`<script type="module">`).

2.3 Ce qui ne change pas : la spécification

Les trois fichiers `.feature` ont été déplacés sans être modifiés (Git les voit comme de simples renommages). Ce sont les mêmes 24 scénarios, maintenant exécutés par Cucumber-js. Qu'ils restent verts après la réécriture prouve que le comportement n'a pas changé : **la spécification exécutable sert de filet de sécurité pour la migration**. Les définitions d'étapes, le page object et l'oracle IndexedDB ont été traduits de Java en JavaScript un pour un.

3. L'application To-Do

3.1 Fonctionnalités

- **Projets** : créer un projet (nom obligatoire, doublons refusés sans tenir compte de la casse ni des espaces), le supprimer avec ses tâches, filtrer les tâches par projet ;
- **Tâches** : ajouter une tâche dans un projet (titre et projet obligatoires), la **modifier** (titre et/ou projet), la terminer ou la réouvrir, la supprimer ;
- **Compteur** : « *n* tâches, dont *k* terminées », ou « Aucune tâche » ;
- **Persistance** : tout est enregistré dans IndexedDB et retrouvé après un rechargement ou dans un nouvel onglet.

L'application n'utilise aucune bibliothèque : HTML, CSS et JavaScript standard. Les textes saisis sont insérés avec `textContent`, jamais `innerHTML` : pas d'injection HTML.

3.2 Modèle de données IndexedDB

Magasin	Champs	Index
`projets`	`id` (auto), `nom`, `nomNormalise`, `creeLe`	`nomNormalise` **unique** : refuse les doublons
`taches`	`id` (auto), `titre`, `projetId`, `terminee`, `creeLe`, `modifieLe`	`projetId` : tâches d'un projet

La base s'appelle `todo-inf563` (version 1). Ce schéma est un contrat : l'oracle des scénarios et les outils de développement le lisent sans passer par l'application. Un test unitaire le vérifie désormais (section 7.4).

3.3 Organisation du code

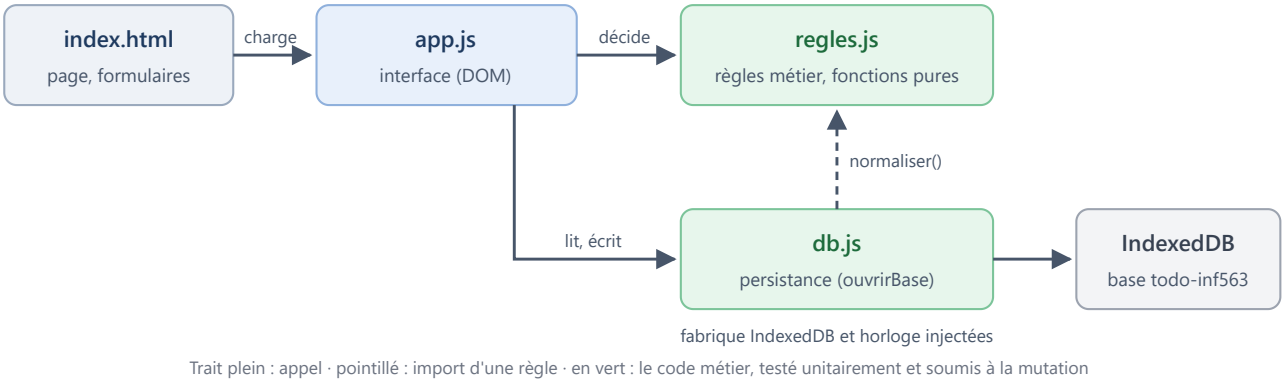


Figure 1 – Organisation du code : l'interface délègue les décisions à `regles.js` et la persistance à `db.js`.

4. Stratégie de test

4.1 Trois niveaux, deux mesures

Niveau	Outils	Tests	Ce qui est vérifié	Mesures appliquées
Unitaire : règles métier	Vitest	37	chaque règle de `regles.js`, valeurs limites et classes d'entrées comprises	couverture Istanbul, mutation StrykerJS
Unitaire : persistance	Vitest, fake-indexeddb, horloge fixe	14	`db.js` : schéma, ajouts, cascade, erreurs, dates	couverture Istanbul, mutation StrykerJS
Acceptation (BDD)	Cucumber-js, Playwright, Google Chrome	24 scénarios	chaque fonctionnalité, de bout en bout, sur l'écran et dans IndexedDB	couverture Istanbul dans le navigateur, rapport nyc

C'est la pyramide des tests : beaucoup de tests unitaires rapides (51 en un quart de seconde) à la base, peu de scénarios de bout en bout (24 en une dizaine de secondes) au sommet. Le cours le recommande : *« couverture de branches en quality gate sur tout le projet, score de mutation sur les modules à risque »*. La mutation porte donc sur `regles.js` et `db.js`, la couverture sur toute l'application.

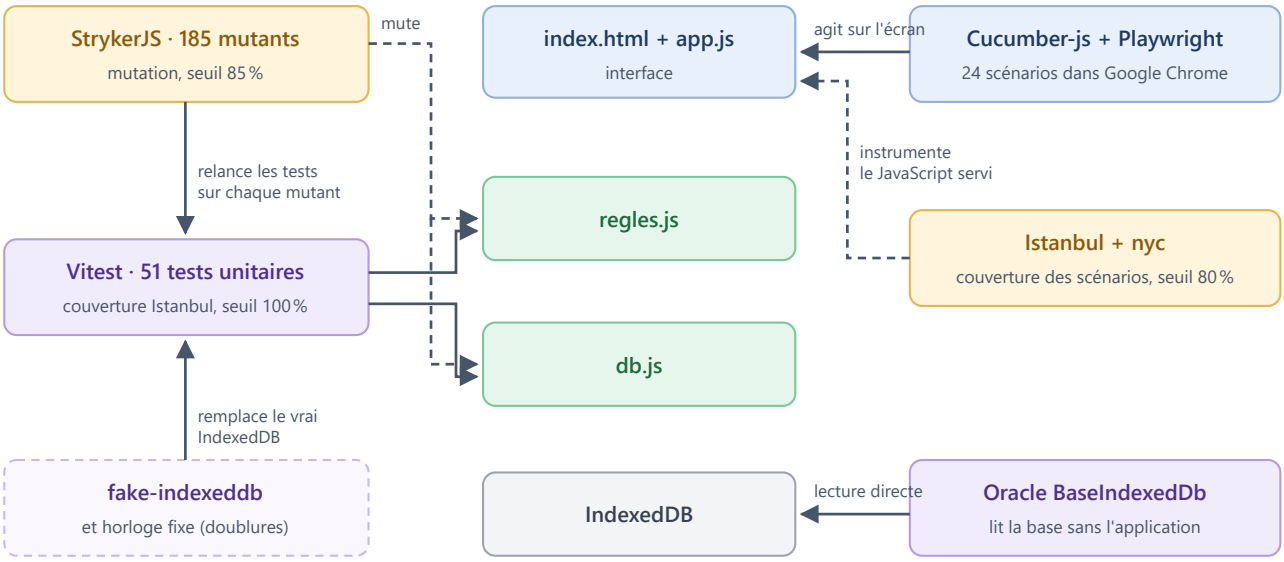


Figure 2 – Qui teste quoi, et ce qui est mesuré. À gauche, les tests unitaires et la mutation du code métier ; à droite, les scénarios de bout en bout et leur couverture de toute l'application.

4.2 Arborescence

```
inf563-bdd-todo-indexeddb/
├── .github/workflows/ci.yml       pipeline : couverture, mutation, BDD à chaque push
├── package.json                   scripts npm et dépendances (package-lock.json : versions figées)
├── vitest.config.js               tests unitaires, couverture Istanbul et ses seuils
├── stryker.config.mjs             test de mutation StrykerJS et son seuil
├── cucumber.mjs, .nycrc.json      scénarios BDD ; rapport et seuils de leur couverture
├── src/                           l'application, servie telle quelle au navigateur
│   ├── index.html, css/style.css
│   └── js/regles.js, js/db.js, js/app.js
├── tests/unitaires/               regles.test.js, db.test.js (suites T1 puis T2)
├── features/                      01_projets, 02_taches, 03_persistance_indexeddb (.feature)
│   ├── step_definitions/          todo.steps.js : phrases Gherkin → actions et assertions
│   └── support/                   monde.js (hooks), page-todo.js (page object),
```

```
│                              base-indexeddb.js (oracle), couverture.js (Istanbul)
├── outils/serveur.js          serveur HTTP (npm start et scénarios)
└── docs/rapports/             instantané des rapports ; docs/v1-java/ : rapport de la version Java
```

4.3 Bibliothèques

Paquet npm	Version	Rôle
`vitest`	5.1.0	exécute les tests unitaires (modules ES natifs)
`@vitest/coverage-istanbul`	5.1.0	couverture Istanbul des tests unitaires, seuils
`@stryker-mutator/core`, `vitest-runner`	10.1.1	test de mutation, piloté par Vitest
`fake-indexeddb`	6.2.8	doublure d'IndexedDB en mémoire pour les tests de `db.js`
`@cucumber/cucumber`	13.3.0	exécute les scénarios Gherkin en français
`@playwright/test`	1.65.0	pilote Chromium ou Google Chrome ; assertions qui attendent automatiquement
`istanbul-lib-instrument`, `istanbul-lib-coverage`	6.0.3 / 3.2.2	instrumentent le JavaScript servi au navigateur et fusionnent la couverture des scénarios
`nyc`	18.0.0	rapport et seuils de couverture des scénarios

Ce sont uniquement des dépendances de développement : l'application livrée n'en a aucune.

5. Tests unitaires et doublures

5.1 Règles métier : des fonctions pures

Les fonctions de `regles.js` ne dépendent que de leurs paramètres : un test leur donne une entrée et compare la sortie à l'attendu, sans navigateur ni base. La première suite, **T1**, est écrite comme le ferait un développeur qui vise la couverture : **un test par chemin**. Par exemple, pour la validation d'un nom de projet :

tests/unitaires/regles.test.js · lignes 17 à 29

```
describe('validerNomProjet', () => {
  it('refuse un nom vide', () => {
    expect(validerNomProjet('', [])).toEqual({ ok: false, message: MESSAGES.NOM_PROJET_OBLIGATOIRE });
  });

  it('refuse un doublon en citant le projet existant', () => {
    expect(validerNomProjet('maison', [maison])).toEqual({ ok: false, message: 'Le projet « Maison » existe
déjà.' });
  });

  it('accepte un nouveau nom', () => {
    expect(validerNomProjet('Travail', [maison])).toEqual({ ok: true, nom: 'Travail' });
  });
});
```

Ces trois tests passent par les trois branches de `validerNomProjet`. La section 7 montre ce qu'ils ne vérifient pas : le premier compare le message à la constante `MESSAGES`, c'est-à-dire au code lui-même.

5.2 Persistance : une *fake* et une horloge injectée

Tester `db.js` demande un IndexedDB. Node.js n'en a pas ; la bibliothèque **fake-indexeddb** en fournit une implémentation complète, en mémoire. Dans la classification des doublures vue en cours, c'est une ***fake*** : une vraie implémentation, simplifiée, et non un *stub* qui renverrait des réponses préparées. Chaque test reçoit une fabrique neuve, donc une base vide : les tests sont isolés et peuvent s'exécuter dans n'importe quel ordre.

tests/unitaires/db.test.js · lignes 1 à 13

```
// Doublure de type « fake » : fake-indexeddb est une vraie implémentation d'IndexedDB, en
// mémoire. Une fabrique neuve par test : chaque test part d'une base vide (isolation).
import { IDBFactory } from 'fake-indexeddb';
import { afterEach, beforeEach, describe, expect, it, vi } from 'vitest';
import { ouvrirBase } from '../../src/js/db.js';

let base;

beforeEach(async () => {
  base = await ouvrirBase({ fabrique: new IDBFactory() });
});

afterEach(() => base.fermer());
```

L'horloge est injectée de la même façon (« maîtriser le temps », comme avec `Clock.fixed` en Java) : le test fixe l'heure, puis vérifie les dates exactes de création et de modification.

tests/unitaires/db.test.js · lignes 122 à 142

```
describe('T2 · horloge injectée', () => {
  // Tue « maintenant » → « () => undefined » : les dates n'étaient jamais vérifiées.
  // L'horloge est une doublure contrôlée par le test (cours, séance 9 : maîtriser le temps).
  it('date la création, puis la modification sans toucher à la date de création', async () => {
    const creation = '2026-11-11T08:00:00.000Z';
    const modification = '2026-11-11T09:30:00.000Z';
    let maintenant = new Date(creation);
    const datee = await ouvrirBase({ fabrique: new IDBFactory(), horloge: () => maintenant });

    const maison = await datee.ajouterProjet('Maison');
    const id = await datee.ajouterTache({ titre: 'Laver la voiture', projetId: maison });
    maintenant = new Date(modification);
    await datee.modifierTache(id, { terminee: true });

    expect(await datee.listerProjets()).toEqual([{ id: maison, nom: 'Maison', nomNormalise: 'maison', creeLe:
creation }]);
    expect(await datee.listerTaches()).toEqual([
      { id, titre: 'Laver la voiture', projetId: maison, terminee: true, creeLe: creation, modifieLe:
modification },
    ]);
    datee.fermer();
  });
});
```

6. Couverture de code avec Istanbul

6.1 Mise en place : l'équivalent de JaCoCo

Au chapitre V, JaCoCo s'installe dans le `pom.xml` en trois buts : `prepare-agent` instrumente le bytecode pendant les tests, `report` produit le rapport HTML et `check` fait échouer le build sous un seuil. Avec Vitest, la même chose tient dans `vitest.config.js` : le fournisseur `istanbul` instrumente le code JavaScript, les `reporter` produisent les rapports et `thresholds` joue le rôle de `check`.

vitest.config.js · lignes 1 à 20

```
import { defineConfig } from 'vitest/config';

// Tests unitaires (Vitest) et couverture de code (Istanbul), l'équivalent JavaScript de
// JUnit 5 + JaCoCo : « npm run test:couverture » échoue sous les seuils (quality gate).
export default defineConfig({
  test: {
    include: ['tests/unitaires/**/*.test.js'],
    environment: 'node',
    coverage: {
      provider: 'istanbul',
      include: ['src/js/regles.js', 'src/js/db.js'],
      reporter: ['text', 'html', 'json-summary'],
      reportsDirectory: 'rapports/couverture-unitaire',
      // Code critique : 100 % atteint (objectif du TP), le seuil interdit toute régression.
      // La gate générale du cours (80 % de branches) s'applique à toute l'application,
      // mesurée sur les scénarios BDD (.nycrc.json).
      thresholds: { branches: 100, lines: 100, functions: 100, statements: 100 },
    },
  },
});
```

`npm run test:couverture` exécute les tests et produit `rapports/couverture-unitaire/index.html`. Si une seule mesure passe sous son seuil, la commande échoue, et le pipeline avec elle : c'est la *quality gate* du cours.

6.2 Lire un rapport Istanbul

Compteur Istanbul	Équivalent JaCoCo	Ce qu'il mesure
Statements	Instructions	instructions JavaScript exécutées au moins une fois
Branches	Branches	chaque issue de `if`, `? :`, `&&`, `||`, `??` et des paramètres par défaut
Functions	Méthodes	fonctions appelées au moins une fois (fonctions fléchées comprises)
Lines	Lignes	lignes contenant au moins une instruction exécutée

Dans le code source annoté, une ligne rouge n'a jamais été exécutée ; les marqueurs `I` et `E` signalent un `if` dont la branche « alors » ou « sinon » n'a jamais été prise. Comme les lignes jaunes de JaCoCo, ce sont les plus instructives. Particularité de JavaScript : Istanbul compte chaque **paramètre par défaut** comme une branche, car la valeur par défaut n'est calculée que si l'argument manque.

6.3 Résultats : T1 puis T2

Suite	Tests	Instructions	Branches	Fonctions	Lignes
T1 · `regles.js`	23	100 %	96,15 % (25/26)	100 %	100 %
T1 · `db.js`	7	94,82 %	60 % (6/10)	95,83 %	96,07 %
T1 · total	**30**	97,02 % (98/101)	**86,11 % (31/36)**	97,67 % (42/43)	97,5 % (78/80)
T2 · total	**51**	**100 %** (101/101)	**100 %** (36/36)	**100 %** (43/43)	**100 %** (80/80)

Avec T1, la couverture dépasse déjà le seuil du cours (80 % des branches). Le rapport désigne pourtant des trous précis :

- dans `regles.js`, la branche du **singulier** de `pluriel` n'est jamais prise : T1 ne teste que 2 et 3 ;
- dans `db.js`, quatre branches sur dix manquent : la modification d'une **tâche inexistante**, l'**AbortError** qui en découle, et deux paramètres par défaut (appel de `ouvrirBase()` sans options, fabrique du navigateur).

Avec T2, les quatre mesures atteignent 100 %. Le seuil de ces deux modules passe alors à 100 % : une branche ajoutée sans test fera désormais échouer le pipeline.

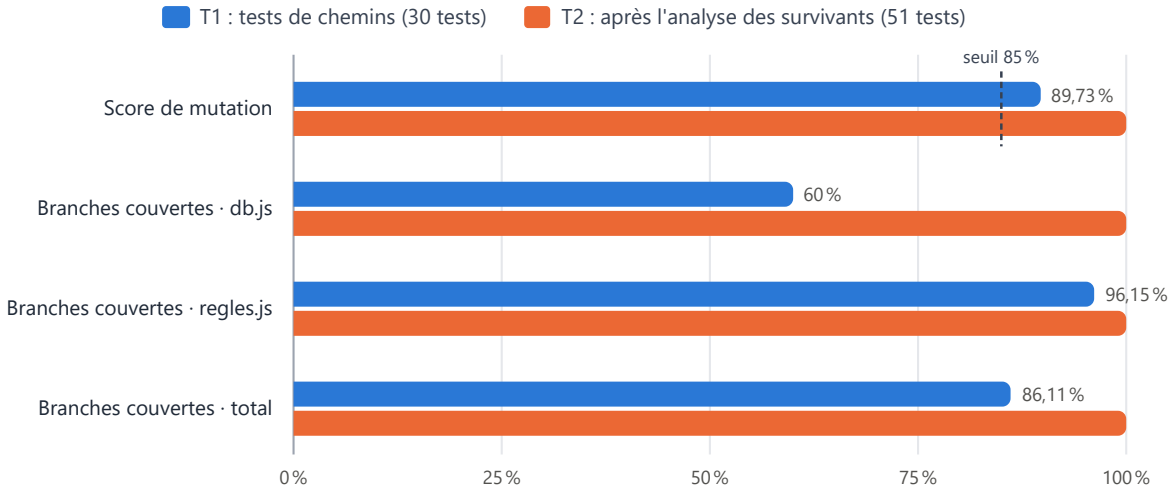


Figure 3 – Score de mutation et couverture des branches, avant et après l'analyse des survivants. T2 atteint 100 % sur les quatre mesures ; les valeurs exactes sont dans les tableaux des sections 6.3 et 7.3.

6.4 Ce que la couverture ne dit pas

> **100 % des lignes, et pourtant sept fautes non détectées**
>
> Avec T1, `regles.js` était couvert à 100 % des instructions et des lignes. Le test de mutation y a pourtant laissé **sept mutants vivants**, dont trois messages d'erreur remplacés par une chaîne vide. Le cours le résume ainsi : la couverture mesure ce qui est **exécuté**, pas ce qui est **vérifié**. Une couverture basse prouve que des tests manquent ; une couverture haute ne prouve rien sur leur qualité. D'où le test de mutation.

7. Test de mutation avec StrykerJS

7.1 Principe et mise en place : l'équivalent de PIT

Le test de mutation évalue les tests en injectant volontairement de petites fautes dans le code. Il repose sur les deux hypothèses du cours (DeMillo, Lipton et Sayward, 1978) : le **programmeur compétent** écrit un code proche du code correct, et l'**effet de couplage** veut qu'une suite qui détecte les fautes simples détecte la plupart des fautes complexes. Chaque version fautive est un **mutant** ; on relance les tests sur chacun. Si un test échoue, le mutant est **tué** ; sinon il **survit**, et il manque un test, à moins que le mutant soit **équivalent** (même comportement).

stryker.config.mjs

```
// Test de mutation (StrykerJS), l'équivalent JavaScript de PIT :
// « npm run test:mutation » injecte des fautes dans le code métier, relance les tests
// unitaires sur chaque mutant, et échoue sous 85 % (comme <mutationThreshold>85</mutationThreshold>).
/** @type {import('@stryker-mutator/api/core').PartialStrykerOptions} */
export default {
  testRunner: 'vitest',
  vitest: { configFile: 'vitest.config.js' },
  // Cibler le code à risque (règles métier, persistance), pas l'affichage : comme <targetClasses>
  mutate: ['src/js/regles.js', 'src/js/db.js'],
  // Chaque mutant n'est confronté qu'aux tests qui couvrent sa ligne (comme PIT)
  coverageAnalysis: 'perTest',
  reporters: ['html', 'json', 'clear-text', 'progress'],
  htmlReporter: { fileName: 'rapports/mutation/index.html' },
  jsonReporter: { fileName: 'rapports/mutation/mutation.json' },
  thresholds: { high: 90, low: 85, break: 85 },
  tempDirName: '.stryker-tmp',
};
```

Comme PIT, StrykerJS cible le code à risque (`mutate` ≈ `<targetClasses>`) et fait échouer le build sous 85 % (`break` ≈ `<mutationThreshold>`). Deux mécanismes le rendent rapide :

- la **commutation de mutants** : les 185 mutants sont insérés une seule fois dans une copie instrumentée du code, chacun derrière un interrupteur ; Stryker active un mutant à la fois, sans recompiler ;
- l'**analyse de couverture par test** (`perTest`) : une première exécution note quels tests couvrent chaque mutant, puis chaque mutant ne lance que ceux-là (1,99 test par mutant en moyenne avec T1, 2,48 avec T2). PIT procède de même.

Résultat : les 185 mutants sont jugés en **11 secondes** sur le poste de développement et en 26 secondes en CI.

Statut Stryker	Signification	Dans le score
Killed (tué)	au moins un test échoue sur le mutant	détecté
Timeout	les tests ne se terminent plus (boucle infinie, promesse jamais résolue)	détecté, comme `TIMED_OUT` chez PIT
Survived (survivant)	tous les tests passent malgré la faute	non détecté
NoCoverage	aucun test n'exécute la ligne mutée	non détecté
Ignored	mutant écarté par un commentaire `// Stryker disable … : raison`	exclu : c'est ainsi qu'on justifie par écrit un mutant équivalent

Score de mutation = (tués + timeouts) ÷ (mutants − ignorés − erreurs). C'est la formule du cours, tués ÷ (générés − équivalents), où les équivalents sont les mutants ignorés avec leur justification. Ici, aucun mutant n'a eu besoin d'être ignoré.

7.2 Les opérateurs de mutation

StrykerJS a généré 185 mutants sur `regles.js` (107) et `db.js` (78). Ses opérateurs recoupent ceux de PIT présentés au chapitre V, plus quelques-uns propres à JavaScript :

Opérateur StrykerJS	Faute simulée	Équivalent PIT	Mutants
StringLiteral	une chaîne devient `""` (une chaîne vide devient `"Stryker was here!"`)	`EMPTY_RETURNS` (chaînes)	46
ConditionalExpression	une condition devient `true` ou `false`	`REMOVE_CONDITIONALS` (hors défauts)	35
BlockStatement	le corps d'une fonction ou d'un bloc est vidé	pas d'équivalent direct	27

ArrowFunction	une fonction fléchée devient `() => undefined`	`NULL_RETURNS`	18
ObjectLiteral	un objet devient `{}`	`EMPTY_RETURNS`	15
EqualityOperator	`>` → `>=` ou `<=`, `===` → `!==`	`CONDITIONALS_BOUNDARY`, `NEGATE_CONDITIONALS`	15
BooleanLiteral	`true` ↔ `false`, un `!` est retiré	`TRUE_RETURNS`, `FALSE_RETURNS`	13
MethodExpression	un appel de méthode est retiré ou remplacé (`trim()` supprimé…)	aucun (propre à JavaScript)	8
LogicalOperator	`&&` ↔ `||`, `??` → `&&`	aucun par défaut	3
CallExpression	un appel est supprimé	`VOID_METHOD_CALLS`	3
ArrayDeclaration	un tableau devient `[]`	`EMPTY_RETURNS`	1
OptionalChaining	`?.` devient `.`	aucun (propre à JavaScript)	1

Les opérateurs arithmétiques (`MATH`, `INCREMENTS`, `INVERT_NEGS` chez PIT ; `ArithmeticOperator`, `UpdateOperator`, `UnaryOperator` chez Stryker) n'ont rien produit : ce code ne fait aucun calcul.

7.3 Suite T1 : 89,73 %

Module	Mutants	Tués	Timeout	Survivants	Sans couverture	Score
`regles.js`	107	99	0	7	1	92,52 %
`db.js`	78	65	2	7	4	85,90 %
Total T1	**185**	164	2	**14**	**5**	**89,73 %**
Total T2	**185**	183	2	**0**	**0**	**100 %**

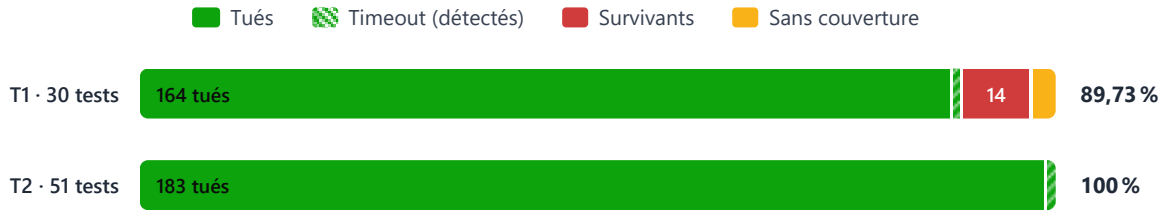


Figure 4 – Devenir des 185 mutants avec chaque suite. Les petits segments (2 timeouts, 5 mutants sans couverture) sont détaillés dans le tableau ci-dessus.

Le score de T1 dépasse déjà le seuil de 85 % : le build passerait. Le TP demande pourtant d'analyser **chaque** survivant, car chacun signale soit une entrée ou une assertion manquante, soit un mutant équivalent à justifier.

7.4 Analyse des survivants

#	Lieu	Opérateur	Mutant	T1	Pourquoi il survivait	Test T2 qui le tue
1	`regles.js:64`	EqualityOperator	`nombre > 1` → `nombre >= 1`	survit	T1 ne teste que 2 et 3 : la **valeur limite 1** n'est jamais essayée	`pluriel(1)` donne « 1 terminée »
2	`regles.js:64`	ConditionalExpression	`nombre > 1` → `true`	survit	idem	`pluriel(0)`, `pluriel(1)`
3	`regles.js:64`	StringLiteral	`''` → `"Stryker was here!"`	non couvert	la branche du singulier n'est jamais exécutée	compteur « 1 tâche, dont 0 terminée »

4	`regles.js:24`	MethodExpression	`nom.trim()` → `nom`	survit	la **classe « espaces »** n'est jamais essayée	`' '` refusé ; `' Travail '` donne `'Travail'`
5	`regles.js:33`	MethodExpression	`titre.trim()` → `titre`	survit	idem pour le titre	titre fait d'espaces refusé, titre nettoyé
6	`regles.js:8`	StringLiteral	message « nom obligatoire » → `""`	survit	**oracle dépendant du code** : T1 compare à la constante `MESSAGES`, qui mute en même temps	texte exact de la spécification (`.feature`)
7	`regles.js:9`	StringLiteral	message « titre obligatoire » → `""`	survit	idem	idem
8	`regles.js:10`	StringLiteral	message « choisissez un projet » → `""`	survit	idem	idem
9	`db.js:13`	StringLiteral	`NOM_BASE` → `""`	survit	le nom de la base n'est jamais vérifié	schéma : la base `todo-inf563` est ouverte directement
10	`db.js:35`	StringLiteral	nom de l'index `nomNormalise` → `""`	survit	l'unicité porte sur le champ, pas sur le nom de l'index : rien ne change pour `db.js`	schéma : magasins, clés et index documentés
11	`db.js:19`	ArrowFunction	`onerror` de l'ouverture → `() => undefined`	survit	aucun échec d'ouverture n'est testé	`VersionError` si la base est plus récente
12	`db.js:49`	ArrowFunction	`maintenant` → `() => undefined`	survit	les dates `creeLe` et `modifieLe` ne sont jamais vérifiées	**horloge injectée** : dates exactes
13	`db.js:72`	CallExpression	`curseur.result.continue()` supprimé	survit	la boucle de suppression n'est testée qu'avec **une** tâche	cascade sur 2 tâches, plus une tâche d'un autre projet
14	`db.js:92`	ConditionalExpression	`!lecture.result` → `false`	survit	aucune modification d'une tâche inexistante	`modifierTache(999)` rejette et n'écrit rien
15	`db.js:92`	BlockStatement	`{ tx.abort(); return; }` → `{}`	non couvert	idem	idem
16	`db.js:93`	CallExpression	`tx.abort()` supprimé	non couvert	idem	idem
17	`db.js:29`	StringLiteral	`'Transaction annulée'` → `""`	non couvert	idem	message de l'erreur vérifié
18	`db.js:29`	StringLiteral	`'AbortError'` → `""`	non couvert	idem	nom de l'erreur vérifié
19	`db.js:107`	ArrowFunction	`fermer` → `() => undefined`	survit	la fermeture de la connexion n'est jamais vérifiée	après `fermer()`, une lecture lève `InvalidStateError`

Aucun mutant équivalent. Le n°10 aurait pu passer pour équivalent : renommer l'index ne change rien pour `db.js`, qui ne l'appelle jamais par son nom. Mais le schéma est un contrat documenté, lu par l'oracle des scénarios et par les outils de développement ; le vérifier est légitime, et le test qui le fait tue aussi le n°9. Les **2 timeouts** (`onsuccess` → `() => undefined`, corps de la promesse vidé) laissent une promesse qui ne se termine jamais : le test dépasse son délai, et Stryker compte ces mutants comme détectés.

7.5 Suite T2 : 100 %

T2 ajoute 21 tests (14 sur `regles.js`, 7 sur `db.js`). Chacun indique en commentaire le mutant qu'il tue, pour que le lien entre l'analyse et le test reste lisible dans le code. Trois exemples :

Valeurs limites du pluriel (survivants 1 à 3)

tests/unitaires/regles.test.js · lignes 125 à 138

```
describe('T2 · valeurs limites du pluriel', () => {
  // Tue « nombre > 1 » → « nombre >= 1 » et → « true » : il faut tester la frontière 1
  it.each([
    [0, '0 terminée'],
    [1, '1 terminée'],
    [2, '2 terminées'],
  ])('pluriel(%i) donne « %s »', (nombre, attendu) => {
    expect(pluriel(nombre, 'terminée')).toBe(attendu);
  });

  it('le compteur est au singulier pour une seule tâche', () => {
    expect(texteCompteur([tache(1, 1)])).toBe('1 tâche, dont 0 terminée');
  });
});
```

Un oracle indépendant du code (survivants 6 à 8)

tests/unitaires/regles.test.js · lignes 159 à 167

```
describe('T2 · oracle indépendant du code', () => {
  // Les tests T1 comparaient aux constantes MESSAGES : muter une constante changeait aussi
  // l'attendu. L'oracle est ici le texte exact de la spécification (fichiers .feature).
  it('les messages d\'erreur sont ceux de la spécification', () => {
    expect(validerNomProjet('', []).message).toBe('Le nom du projet est obligatoire.');
    expect(validerTitre('').message).toBe('Le titre de la tâche est obligatoire.');
    expect(validerNouvelleTache('Acheter du pain', null).message).toBe('Choisissez un projet pour la tâche.');
  });
});
```

La boucle testée avec plusieurs tâches (survivant 13)

tests/unitaires/db.test.js · lignes 144 à 157

```
describe('T2 · suppression en cascade et cas d\'erreur', () => {
  // Tue « curseur.result.continue() » supprimé : la boucle doit être testée avec plusieurs tâches
  it('supprimerProjet supprime toutes les tâches du projet, et seulement elles', async () => {
    const maison = await base.ajouterProjet('Maison');
    const travail = await base.ajouterProjet('Travail');
    await base.ajouterTache({ titre: 'Faire les courses', projetId: maison });
    await base.ajouterTache({ titre: 'Écrire le rapport', projetId: travail });
    await base.ajouterTache({ titre: 'Ranger le garage', projetId: maison });

    await base.supprimerProjet(maison);

    expect((await base.listerProjets()).map((p) => p.nom)).toEqual(['Travail']);
    expect((await base.listerTaches()).map((t) => t.titre)).toEqual(['Écrire le rapport']);
  });
```

Avec T2 : 185 mutants, 183 tués, 2 en timeout, **0 survivant, score 100 %**. Les rapports des deux exécutions sont versionnés : `docs/rapports/mutation-T1/index.html` et `docs/rapports/mutation-T2/index.html`.

7.6 Ce que montre la comparaison T1 / T2

Trois leçons du chapitre V, observées sur ce projet

- **Les mutants de limite survivent quand les tests évitent les frontières.** T1 testait le pluriel avec 2 et 3 ; seule la valeur 1 distingue `> 1` de `>= 1`. La mutation mesure la qualité de l'analyse des valeurs limites.
- **Un oracle tiré du code ne détecte rien.** Comparer un message à sa propre constante est une tautologie : muter la constante change aussi l'attendu. L'oracle de T2 est le texte de la spécification.
- **Les survivants révèlent les entrées et les assertions manquantes.** Une boucle testée une seule fois, des dates jamais lues, un cas d'erreur jamais provoqué : autant de comportements que T1 exécutait parfois, sans jamais les vérifier.

8. Défaut détecté par les tests unitaires

Le premier test de T1 sur les doublons a échoué : ajouter le projet « MAISON » quand « Maison » existe rejetait la promesse avec `null` au lieu d'une `ConstraintError`. Voici le code d'origine, repris tel quel de la version 1 :

src/js/db.js · lignes 23 à 29 · version d'origine (commit 46a1db4)

```
function terminee(transaction) {
  return new Promise((resoudre, rejeter) => {
    transaction.oncomplete = () => resoudre();
    transaction.onerror = () => rejeter(transaction.error);
    transaction.onabort = () => rejeter(transaction.error || new Error('Transaction annulée'));
  });
}
```

Dans IndexedDB, une requête en échec déclenche d'abord l'événement `error`, qui remonte jusqu'à la transaction ; la transaction est ensuite annulée, et c'est seulement à ce moment que `transaction.error` est renseignée. Dans `onerror`, elle vaut encore `null` : la promesse rejetait donc avec `null`. Conséquence : le filet de sécurité de `app.js`, qui reconnaît un doublon refusé par la base (`erreur?.name === 'ConstraintError'`, utile si deux onglets créent le même projet), ne pouvait jamais fonctionner.

La correction rejette dans `onabort`, où l'erreur est connue, et donne un nom explicite à l'annulation volontaire :

src/js/db.js · lignes 23 à 31

```
/** Se résout quand la transaction est validée ; rejette avec l'erreur qui l'a annulée. */
function terminee(transaction) {
  return new Promise((resoudre, rejeter) => {
    transaction.oncomplete = () => resoudre();
    // Une requête en échec annule la transaction : transaction.error n'est renseignée qu'à
    // l'annulation (elle vaut encore null dans onerror, d'où l'ancien rejet avec null).
    transaction.onabort = () => rejeter(transaction.error ?? new DOMException('Transaction annulée',
'AbortError'));
  });
}
```

Pourquoi les scénarios BDD ne l'avaient pas vu

L'écran refuse le doublon **avant** d'écrire, avec la règle de `regles.js` : en passant par l'interface, la base ne voit jamais de doublon. Seul un test qui appelle `db.js` directement atteint ce chemin. C'est la complémentarité des niveaux de la pyramide : les scénarios prouvent le comportement de bout en bout, les tests unitaires atteignent les chemins d'erreur que l'interface ne provoque pas. L'historique Git garde la trace des deux étapes : « TDD (rouge) » puis « TDD (vert) ».

9. Scénarios BDD en JavaScript

9.1 Cucumber-js et Playwright

Les 24 scénarios des trois fonctionnalités (*Organiser mes tâches par projet*, *Gérer les tâches d'un projet*, *Sauvegarde dans IndexedDB*) sont exécutés par Cucumber-js, qui relie chaque phrase Gherkin à une définition d'étape. Le fichier `features/support/monde.js` organise leur exécution :

- **une fois pour toute la suite** (`BeforeAll`) : démarrer le serveur HTTP local sur un port libre (IndexedDB exige une vraie origine `http://`) et lancer le navigateur ;
- **avant chaque scénario** (`Before`) : ouvrir un **contexte de navigation neuf**, donc un IndexedDB vide. Les scénarios sont isolés et rejouables dans n'importe quel ordre (F.I.R.S.T.) ;
- **après chaque scénario** (`After`) : joindre une capture d'écran au rapport en cas d'échec, relever la couverture, fermer le contexte, et faire échouer le scénario si la page a levé une erreur JavaScript.

Les étapes n'expriment que des intentions ; tous les sélecteurs (libellés, rôles ARIA, `data-testid`) vivent dans le page object `features/support/page-todo.js` . Les assertions de Playwright **attendent automatiquement** que l'état attendu apparaisse : aucune pause fixe. L'étape « je coche la tâche » montre pourquoi le point de synchronisation compte :

features/step_definitions/todo.steps.js · lignes 89 à 96

```
// La case est cochée par le navigateur AVANT l'écriture dans IndexedDB : ce n'est pas une
// preuve de sauvegarde. data-terminee n'est rendu qu'après relecture de la base : c'est le
// point de synchronisation (sinon un rechargement immédiat perd l'écriture).
When('je coche la tâche {string}', async function (titre) {
  await expect(this.todo.tache(titre)).toHaveAttribute('data-terminee', 'false');
  await this.todo.basculerTerminee(titre);
  await expect(this.todo.tache(titre)).toHaveAttribute('data-terminee', 'true');
});
```

Ce point de synchronisation vient d'un défaut trouvé par la version 1 : le navigateur coche la case avant la fin de l'écriture dans IndexedDB, et un rechargement immédiat perdait la modification. Les 24 scénarios s'exécutent en environ **10 secondes**, deux fois plus vite que la version Java (19,8 secondes).

9.2 Deux oracles : l'écran et la base

Vérifier l'écran ne suffit pas : une tâche affichée n'est pas forcément sauvegardée. L'oracle `features/support/base-indexeddb.js` lit IndexedDB **directement**, avec l'API du navigateur, sans passer par le code de l'application. Une étape comme « la base IndexedDB contient exactement les tâches : » compare ce contenu réel au tableau Gherkin avec `assert.deepEqual` . L'oracle sait aussi pré-remplir la base, pour le scénario qui vérifie que l'application affiche des données déjà présentes.

9.3 Couverture des scénarios

L'interface (`app.js`) n'est exécutée que dans le navigateur : Vitest ne peut pas la mesurer. Le profil Cucumber `couverture` reproduit donc dans le navigateur ce que fait l'agent JaCoCo pendant les tests :

1. le serveur de test **instrumente à la volée** chaque fichier JavaScript qu'il sert (Istanbul) ;
2. le code instrumenté compte ce qu'il exécute dans `window.__coverage__` ;
3. avant de quitter une page (fin de scénario, rechargement, changement d'onglet), la couverture est relevée et fusionnée ;
4. à la fin, **nyc** produit le rapport et applique le seuil du cours, **80 % des branches** (`.nycrc.json`).

features/support/couverture.js · lignes 12 à 20

```
const DOSSIER = '.nyc_output';
const instrumenteur = libInstrumentation.createInstrumenter({ esModules: true, coverageVariable: '__coverage__'
});
const deja = new Map();
const carte = libCouverture.createCoverageMap({});

export function instrumenter(code, chemin) {
  if (!deja.has(chemin)) deja.set(chemin, instrumenteur.instrumentSync(code, chemin));
  return deja.get(chemin);
}
```

Fichier	Instructions	Branches	Fonctions	Lignes
`app.js`	95,19 %	82,14 %	97,05 %	95,6 %
`db.js`	91,37 %	70 %	87,5 %	94,11 %
`regles.js`	100 %	96,15 %	100 %	100 %
Toute l'application	**95,12 %**	**85,93 %**	**94,8 %**	**95,9 %**

Le seuil de 80 % est respecté. Le rapport montre surtout ce que les scénarios n'exécutent **pas**, et pourquoi :

Code non exécuté par les scénarios	Pourquoi	Où c'est vérifié
`app.js` 116-117 : filet de sécurité si IndexedDB refuse un doublon	l'écran refuse le doublon avant d'écrire ; il faudrait deux onglets concurrents	test unitaire : `db.js` rejette avec une `ConstraintError`
`app.js` 193-194 : échec d'ouverture d'IndexedDB	impossible à provoquer dans un navigateur normal	test unitaire : `ouvrirBase` rejette avec une `VersionError`
`db.js` 93-94 et 107 : tâche inexistante, `fermer()`	inaccessibles depuis l'écran	tests unitaires de T2
`regles.js` 77 : projet introuvable (« ? »)	une tâche ne peut pas perdre son projet par l'interface	test unitaire de `nomDuProjet`

Les deux mesures se répondent : tout ce que les scénarios laissent de côté est un chemin d'erreur, et chacun est couvert par un test unitaire.

10. Intégration continue : chaque push lance les trois mesures

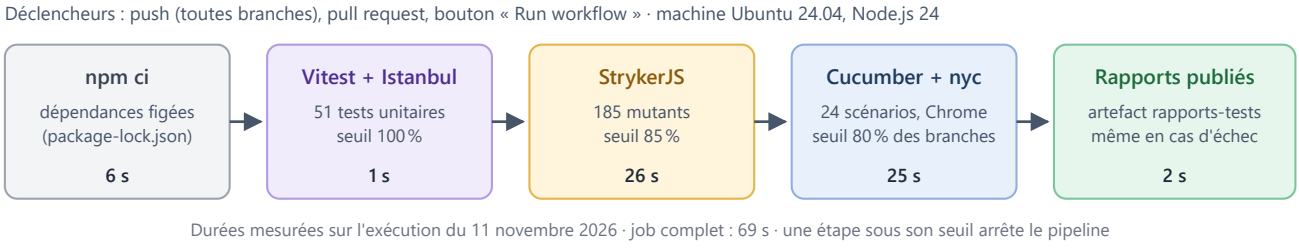


Figure 5 – Le pipeline GitHub Actions (`.github/workflows/ci.yml`), dans l'ordre du cours : tests et couverture, puis mutation, puis scénarios.

Chaque étape porte son seuil dans un fichier du dépôt : `vitest.config.js` pour la couverture unitaire, `stryker.config.mjs` pour la mutation, `.nycrc.json` pour la couverture des scénarios. Une mesure sous son seuil fait échouer l'étape : le pipeline devient rouge sur le commit, le badge du README et la *pull request*, et on ne fusionne pas. Les rapports sont publiés même en cas d'échec (`if: always()`), pour que l'on puisse lire la cause.

Deux choix rendent ce pipeline fiable :

- **une machine figée** (`ubuntu-24.04`) plutôt que `ubuntu-latest`, qui change de version d'Ubuntu sans prévenir : le pipeline reste reproductible ;
- **aucun téléchargement de navigateur** : le 11 novembre 2026, l'étape `playwright install` est restée bloquée 10 minutes avant d'échouer, sans qu'aucun test ait tourné. Les scénarios utilisent désormais le Google Chrome préinstallé sur la machine (`CANAL=chrome`).

11. Résultats

11.1 Synthèse des mesures

Mesure	Outil	Résultat	Seuil	Commande
Tests unitaires	Vitest	51 réussis en 0,25 s	–	`npm run test:unitaires`
Couverture du code métier	Istanbul	**100 %** des instructions, branches, fonctions et lignes	100 %	`npm run test:couverture`
Test de mutation	StrykerJS	**100 %** : 183 tués, 2 timeouts, 0 survivant (T1 : 89,73 %)	85 %	`npm run test:mutation`
Scénarios BDD	Cucumber-js, Playwright	24 réussis, 200 étapes, environ 10 s	–	`npm run test:bdd`
Couverture de l'application par les scénarios	Istanbul, nyc	**85,93 %** des branches, 95,12 % des instructions	80 %	`npm run test:bdd:couverture`
Intégration continue	GitHub Actions	vert en 69 s, aucun avertissement	–	à chaque push

Les mesures ont été obtenues sur le poste de développement (Windows 11, Node.js 24.19, Google Chrome 157) puis en CI (Ubuntu 24.04, Node.js 24, Google Chrome 156), avec les mêmes résultats. `npm run verify` enchaîne les trois mesures comme la CI.

11.2 Historique Git

L'historique suit la démarche du TP ; chaque commit est une étape vérifiable.

Commit	Date	Message
Version 1 : Java (étiquette `v1-java`)		
`6a668a6`	06/11/2026	Projet Maven : Java 21, JUnit 5, Cucumber et Playwright
`541a178`	06/11/2026	Application To-Do : projets et tâches sauvegardés dans IndexedDB
`e23f93a`	06/11/2026	BDD : scénarios Gherkin (projets, tâches, persistance) et étapes Playwright
`3cbbe1c`	06/11/2026	Correctif : synchroniser « je coche la tâche » sur la sauvegarde IndexedDB
`eeabc9a`	06/11/2026	CI : GitHub Actions lance les tests à chaque push
`b49fb13`	06/11/2026	Rapport : choix techniques, scénarios, défaut détecté et lacunes de spécification
`8a1e256`	11/11/2026	Rapport PDF du devoir
`7165b79`	11/11/2026	CI : scénarios dans le Google Chrome préinstallé, sans « playwright install »
Version 2 : JavaScript, couverture et mutation		
`1b15982`	11/11/2026	Migration JavaScript : application en modules ES, scénarios BDD en Cucumber-js + Playwright
`46a1db4`	11/11/2026	TDD (rouge) : tests unitaires T1 (Vitest) et couverture (Istanbul)

`f3d9b6b`	11/11/2026	TDD (vert) : terminee() rejette avec l'erreur qui a annulé la transaction
`7a38370`	11/11/2026	Mutation (StrykerJS) sur T1 : 89,73 %, 14 survivants et 5 mutants non couverts
`cb62f54`	11/11/2026	T2 : 21 tests qui tuent les 19 mutants restants ; mutation 100 %, couverture 100 %
`e8270b5`	11/11/2026	CI Node.js (couverture, mutation, BDD) et README-rapport de la version JavaScript

12. Lacunes de spécification et décisions

Question non tranchée par l'énoncé	Décision
« Maison » et « maison » sont-ils deux projets différents ?	Non : comparaison sans tenir compte de la casse ni des espaces autour (index unique `nomNormalise`).
Que deviennent les tâches d'un projet supprimé ?	Elles sont supprimées avec lui, dans la même transaction (sinon, tâches orphelines).
Un titre fait uniquement d'espaces est-il valide ?	Non : refusé à l'ajout comme à la modification.
Une tâche peut-elle exister sans projet ?	Non : le projet est obligatoire.
Dans quel ordre afficher les tâches ?	Ordre de création (clé auto-incrémentée).
« 0 terminée » ou « 0 terminées » ?	Singulier jusqu'à 1, comme en français : « 0 terminée », « 1 tâche », « 2 tâches ». Cette décision a été rendue explicite par le test de mutation : c'est elle que tranche la frontière `nombre > 1`.

13. Limites et pistes d'amélioration

- `app.js` **n'est pas soumis à la mutation.** Stryker active les mutants dans le processus Node.js des tests, pas dans un navigateur piloté par Playwright. L'interface est vérifiée par les scénarios et mesurée par leur couverture. Piste : des tests unitaires de l'interface dans un DOM simulé (jsdom), que Stryker pourrait alors muter.
- **fake-indexeddb reste une doublure.** Un navigateur réel peut différer sur des détails ; c'est pourquoi les scénarios tournent aussi contre le vrai IndexedDB de Chrome.
- **Un seul moteur de navigateur.** Playwright permet d'ajouter Firefox et WebKit dans `features/support/monde.js`.
- **Coût de la mutation.** 26 secondes en CI aujourd'hui ; si le projet grandit, le mode incrémental de Stryker (`--incremental`) ne rejugerait que les mutants touchés par un changement, comme l'option `withHistory` de PIT.
- **Dépendances de développement.** `npm audit` signale des failles modérées dans des dépendances de nyc et de Stryker (glob, minimatch). Ce sont des outils de test : rien n'est livré avec l'application.
- **Publication des rapports.** GitHub affiche les rapports HTML de `docs/rapports/` comme du code source ; GitHub Pages permettrait de les consulter en ligne.

14. Conclusion

Le devoir 2 est désormais **entièrement en JavaScript**, et les 24 scénarios Gherkin, inchangés, prouvent que la migration n'a pas modifié le comportement. Pour appliquer le chapitre V, le code métier a d'abord été rendu mesurable : règles pures d'un côté, persistance aux dépendances injectées de l'autre. Istanbul joue le rôle de JaCoCo et StrykerJS celui de PIT, chacun avec un seuil qui fait échouer le build.

La démarche du TP a été suivie jusqu'au bout. Une première suite écrite pour la couverture atteignait 86 % des branches et 89,73 % de mutants détectés : assez pour passer les seuils, pas assez pour être fiable. L'analyse des 19 mutants restants a montré à chaque fois une lacune réelle : valeur limite évitée, classe d'entrées oubliée, oracle tiré du

code, comportement exécuté mais jamais vérifié. Les 21 tests ajoutés portent le score de mutation et la couverture à 100 %, sans aucun mutant équivalent à justifier. En chemin, les tests unitaires ont trouvé un défaut réel, invisible depuis l'interface. C'est la leçon du chapitre : la couverture dit ce qui est exécuté, la mutation dit ce qui est vérifié, et les deux ensemble disent où renforcer la suite.

Annexe A – Configuration du projet

Les configurations de Vitest (couverture) et de StrykerJS (mutation) sont reproduites aux sections 6.1 et 7.1.

package.json

```
{
  "name": "inf563-bdd-todo-indexeddb",
  "version": "2.0.0",
  "private": true,
  "description": "INF563 · Devoir 2 : application To-Do (projets, IndexedDB) en JavaScript, testée en BDD (Cucumber + Playwright), avec couverture de code (Istanbul) et test de mutation (StrykerJS)",
  "type": "module",
  "engines": {
    "node": ">=22"
  },
  "scripts": {
    "start": "node outils/serveur.js",
    "test": "npm run test:unitaires && npm run test:bdd",
    "test:unitaires": "vitest run",
    "test:couverture": "vitest run --coverage",
    "test:mutation": "stryker run",
    "test:bdd": "cucumber-js",
    "test:bdd:couverture": "cucumber-js --profile couverture && nyc report && nyc check-coverage",
    "verify": "npm run test:couverture && npm run test:mutation && npm run test:bdd:couverture"
  },
  "devDependencies": {
    "@cucumber/cucumber": "13.3.0",
    "@playwright/test": "1.65.0",
    "@stryker-mutator/core": "10.1.1",
    "@stryker-mutator/vitest-runner": "10.1.1",
    "@vitest/coverage-istanbul": "5.1.0",
    "fake-indexeddb": "6.2.8",
    "istanbul-lib-coverage": "3.2.2",
    "istanbul-lib-instrument": "6.0.3",
    "nyc": "18.0.0",
    "vitest": "5.1.0"
  }
}
```

cucumber.mjs

```
// Configuration de Cucumber-js : « npm run test:bdd » exécute toutes les features.
// Le profil « couverture » (npm run test:bdd:couverture) mesure en plus le code exécuté.
const commun = {
  paths: ['features/**/*.feature'],
  import: ['features/support/**/*.js', 'features/step_definitions/**/*.js'],
  format: [
    'pretty',
    'html:rapports/cucumber/cucumber.html',
    'junit:rapports/cucumber/cucumber.xml',
  ],
};

export default commun;

export const couverture = { ...commun, worldParameters: { couverture: true } };
```

.nycrc.json

```
{
  "temp-dir": ".nyc_output",
  "report-dir": "rapports/couverture-bdd",
  "reporter": ["html", "text", "json-summary"],
  "branches": 80,
  "lines": 80,
  "functions": 80,
  "statements": 80
}
```

Annexe B – Règles métier (src/js/regles.js)

src/js/regles.js

```
/*
 * Règles métier de l'application To-Do : des fonctions pures, sans DOM ni IndexedDB.
 * Elles sont testées unitairement (tests/unitaires/regles.test.js) et soumises au
 * test de mutation (StrykerJS) : c'est le code « à risque » de l'application.
 */

export const MESSAGES = Object.freeze({
  NOM_PROJET_OBLIGATOIRE: 'Le nom du projet est obligatoire.',
  TITRE_OBLIGATOIRE: 'Le titre de la tâche est obligatoire.',
  PROJET_OBLIGATOIRE: 'Choisissez un projet pour la tâche.',
  projetExistant: (nom) => `Le projet « ${nom} » existe déjà.`,
});

/** Clé de comparaison des noms de projet : espaces autour et casse ignorés. */
export function normaliser(texte) {
  return texte.trim().toLocaleLowerCase('fr');
}

/**
 * Valide le nom d'un nouveau projet face aux projets existants.
 * @returns {{ ok: true, nom: string } | { ok: false, message: string }}
 */
export function validerNomProjet(nom, projets) {
  const propre = nom.trim();
  if (propre === '') return { ok: false, message: MESSAGES.NOM_PROJET_OBLIGATOIRE };
  const existant = projets.find((p) => p.nomNormalise === normaliser(propre));
  if (existant) return { ok: false, message: MESSAGES.projetExistant(existant.nom) };
  return { ok: true, nom: propre };
}

/** Valide le titre d'une tâche, à l'ajout comme à la modification. */
export function validerTitre(titre) {
  const propre = titre.trim();
  if (propre === '') return { ok: false, message: MESSAGES.TITRE_OBLIGATOIRE };
  return { ok: true, titre: propre };
}

/** Valide une nouvelle tâche : le titre est vérifié avant le projet. */
export function validerNouvelleTache(titre, projetId) {
  const verdict = validerTitre(titre);
  if (!verdict.ok) return verdict;
  if (projetId === null) return { ok: false, message: MESSAGES.PROJET_OBLIGATOIRE };
  return { ok: true, titre: verdict.titre, projetId };
}

/** Identifiant lu dans une liste déroulante : entier strictement positif, sinon null (aucun choix). */
export function lireIdentifiant(valeur) {
  const nombre = Number(valeur);
  return Number.isInteger(nombre) && nombre > 0 ? nombre : null;
}
```

```
/** Tâches affichées : toutes si aucun projet n'est sélectionné, sinon celles du projet. */
export function tachesVisibles(taches, projetSelectionne) {
  if (projetSelectionne === null) return taches;
  return taches.filter((t) => t.projetId === projetSelectionne);
}

export function nombreDeTaches(taches, projetId) {
  return taches.filter((t) => t.projetId === projetId).length;
}

/** « 0 terminée », « 1 tâche », « 2 tâches » : en français, le pluriel commence à 2. */
export function pluriel(nombre, mot) {
  return `${nombre} ${mot}${nombre > 1 ? 's' : ''}`;
}

/** « Aucune tâche » ou « n tâche(s), dont k terminée(s) ». */
export function texteCompteur(taches) {
  const total = taches.length;
  if (total === 0) return 'Aucune tâche';
  const terminees = taches.filter((t) => t.terminee).length;
  return `${pluriel(total, 'tâche')}, dont ${pluriel(terminees, 'terminée')}`;
}

/** Nom du projet d'une tâche ; « ? » si le projet n'existe plus. */
export function nomDuProjet(projets, tache) {
  return projets.find((p) => p.id === tache.projetId)?.nom ?? '?';
}

/** Un projet sélectionné reste sélectionné seulement s'il existe encore. */
export function selectionValide(projets, projetSelectionne) {
  return projets.some((p) => p.id === projetSelectionne) ? projetSelectionne : null;
}

/**
 * Projet pré-choisi dans le formulaire d'ajout : le projet sélectionné dans la liste,
 * sinon le choix précédent s'il existe encore, sinon aucun.
 */
export function projetParDefaut(projets, projetSelectionne, choixPrecedent) {
  if (projetSelectionne !== null) return projetSelectionne;
  return selectionValide(projets, choixPrecedent);
}
```

Annexe C – Persistance (src/js/db.js)

src/js/db.js

```
/*
 * Accès à IndexedDB : toute la persistance de l'application passe par ce module.
 *
 * Base « todo-inf563 », version 1
 *   projets : { id (auto), nom, nomNormalise (index unique), creeLe }
 *   taches  : { id (auto), titre, projetId (index), terminee, creeLe, modifieLe }
 *
 * La fabrique IndexedDB et l'horloge sont injectées : le navigateur utilise les vraies,
 * les tests unitaires une base en mémoire (fake-indexeddb) et une horloge fixe.
 */
import { normaliser } from './regles.js';

export const NOM_BASE = 'todo-inf563';
export const VERSION = 1;

function resultat(requete) {
  return new Promise((resoudre, rejeter) => {
    requete.onsuccess = () => resoudre(requete.result);
    requete.onerror = () => rejeter(requete.error);
  });
}
```

```
/** Se résout quand la transaction est validée ; rejette avec l'erreur qui l'a annulée. */
function terminee(transaction) {
  return new Promise((resoudre, rejeter) => {
    transaction.oncomplete = () => resoudre();
    // Une requête en échec annule la transaction : transaction.error n'est renseignée qu'à
    // l'annulation (elle vaut encore null dans onerror, d'où l'ancien rejet avec null).
    transaction.onabort = () => rejeter(transaction.error ?? new DOMException('Transaction annulée', 'AbortError'));
  });
}

function creerSchema(db) {
  const projets = db.createObjectStore('projets', { keyPath: 'id', autoIncrement: true });
  projets.createIndex('nomNormalise', 'nomNormalise', { unique: true });
  const taches = db.createObjectStore('taches', { keyPath: 'id', autoIncrement: true });
  taches.createIndex('projetId', 'projetId');
}

/**
 * Ouvre la base (et la crée à la première utilisation).
 * @param {{ fabrique?: IDBFactory, horloge?: () => Date, nom?: string }} [options]
 */
export async function ouvrirBase({ fabrique = globalThis.indexedDB, horloge = () => new Date(), nom = NOM_BASE } = {})
{
  const ouverture = fabrique.open(nom, VERSION);
  ouverture.onupgradeneeded = () => creerSchema(ouverture.result);
  const db = await resultat(ouverture);

  const maintenant = () => horloge().toISOString();
  const lireTout = (store) => resultat(db.transaction(store).objectStore(store).getAll());

  return {
    listerProjets: () => lireTout('projets'),
    listerTaches: () => lireTout('taches'),

    /** Rejette avec une ConstraintError si un projet de même nom (casse ignorée) existe. */
    async ajouterProjet(nom) {
      const tx = db.transaction('projets', 'readwrite');
      const ajout = tx.objectStore('projets').add({ nom, nomNormalise: normaliser(nom), creeLe: maintenant() });
      await terminee(tx);
      return ajout.result;
    },

    /** Supprime le projet et toutes ses tâches dans une seule transaction (tout ou rien). */
    async supprimerProjet(id) {
      const tx = db.transaction(['projets', 'taches'], 'readwrite');
      tx.objectStore('projets').delete(id);
      const curseur = tx.objectStore('taches').index('projetId').openCursor(id);
      curseur.onsuccess = () => {
        if (curseur.result) {
          curseur.result.delete();
          curseur.result.continue();
        }
      };
      await terminee(tx);
    },

    async ajouterTache({ titre, projetId }) {
      const tx = db.transaction('taches', 'readwrite');
      const date = maintenant();
      const ajout = tx.objectStore('taches').add({ titre, projetId, terminee: false, creeLe: date, modifieLe: date });
      await terminee(tx);
      return ajout.result;
    },

    /** Rejette si la tâche n'existe pas : rien n'est écrit. */
    async modifierTache(id, modifications) {
      const tx = db.transaction('taches', 'readwrite');
      const store = tx.objectStore('taches');
```

```
      const lecture = store.get(id);
      lecture.onsuccess = () => {
        if (!lecture.result) {
          tx.abort();
          return;
        }
        store.put({ ...lecture.result, ...modifications, modifieLe: maintenant() });
      };
      await terminee(tx);
    },

    async supprimerTache(id) {
      const tx = db.transaction('taches', 'readwrite');
      tx.objectStore('taches').delete(id);
      await terminee(tx);
    },

    fermer: () => db.close(),
  };
}
```

Annexe D – Tests ajoutés après le test de mutation (suite T2)

Les 30 tests de T1 précèdent ces lignes dans les mêmes fichiers.

tests/unitaires/regles.test.js · lignes 123 à 179

```
// ---- T2 : tests ajoutés après le premier test de mutation (voir le README) ----------

describe('T2 · valeurs limites du pluriel', () => {
  // Tue « nombre > 1 » → « nombre >= 1 » et → « true » : il faut tester la frontière 1
  it.each([
    [0, '0 terminée'],
    [1, '1 terminée'],
    [2, '2 terminées'],
  ])('pluriel(%i) donne « %s »', (nombre, attendu) => {
    expect(pluriel(nombre, 'terminée')).toBe(attendu);
  });

  it('le compteur est au singulier pour une seule tâche', () => {
    expect(texteCompteur([tache(1, 1)])).toBe('1 tâche, dont 0 terminée');
  });
});

describe('T2 · espaces autour des saisies', () => {
  // Tuent « nom.trim() » → « nom » et « titre.trim() » → « titre »
  it('un nom de projet fait d\'espaces est refusé', () => {
    expect(validerNomProjet('   ', []).ok).toBe(false);
  });

  it('un nom de projet est enregistré sans les espaces autour', () => {
    expect(validerNomProjet('  Travail  ', [maison])).toEqual({ ok: true, nom: 'Travail' });
  });

  it('un titre fait d\'espaces est refusé', () => {
    expect(validerTitre('    ').ok).toBe(false);
  });

  it('un titre est enregistré sans les espaces autour', () => {
    expect(validerTitre('  Appeler le plombier ')).toEqual({ ok: true, titre: 'Appeler le plombier' });
  });
});

describe('T2 · oracle indépendant du code', () => {
  // Les tests T1 comparaient aux constantes MESSAGES : muter une constante changeait aussi
  // l'attendu. L'oracle est ici le texte exact de la spécification (fichiers .feature).
  it('les messages d\'erreur sont ceux de la spécification', () => {
```

```
    expect(validerNomProjet('', []).message).toBe('Le nom du projet est obligatoire.');
    expect(validerTitre('').message).toBe('Le titre de la tâche est obligatoire.');
    expect(validerNouvelleTache('Acheter du pain', null).message).toBe('Choisissez un projet pour la tâche.');
  });
});

describe('T2 · valeurs limites de lireIdentifiant', () => {
  it.each([
    ['0', null],
    ['1', 1],
    ['-1', null],
    ['1.5', null],
    ['abc', null],
  ])('lireIdentifiant(%j) donne %j', (valeur, attendu) => {
    expect(lireIdentifiant(valeur)).toBe(attendu);
  });
});
```

tests/unitaires/db.test.js · lignes 62 à 169

```
// ---- T2 : tests ajoutés après le premier test de mutation (voir le README) ----------

const ouvrirSansApplication = (fabrique, nom, version) => new Promise((resoudre, rejeter) => {
  const requete = fabrique.open(nom, version);
  requete.onsuccess = () => resoudre(requete.result);
  requete.onerror = () => rejeter(requete.error);
});

describe('T2 · schéma de la base', () => {
  // Tue NOM_BASE → "" et le nom d'index → "" : le schéma documenté est un contrat,
  // lu par d'autres que ce module (outils de développement, oracle des scénarios BDD)
  it('crée la base todo-inf563 avec les magasins et index documentés', async () => {
    const fabrique = new IDBFactory();
    (await ouvrirBase({ fabrique })).fermer();

    const db = await ouvrirSansApplication(fabrique, 'todo-inf563');
    const tx = db.transaction(['projets', 'taches']);
    const projets = tx.objectStore('projets');
    const taches = tx.objectStore('taches');
    expect(db.version).toBe(1);
    expect([...db.objectStoreNames]).toEqual(['projets', 'taches']);
    expect({ cle: projets.keyPath, auto: projets.autoIncrement, index: [...projets.indexNames] })
      .toEqual({ cle: 'id', auto: true, index: ['nomNormalise'] });
    expect(projets.index('nomNormalise')).toMatchObject({ keyPath: 'nomNormalise', unique: true });
    expect({ cle: taches.keyPath, auto: taches.autoIncrement, index: [...taches.indexNames] })
      .toEqual({ cle: 'id', auto: true, index: ['projetId'] });
    expect(taches.index('projetId')).toMatchObject({ keyPath: 'projetId', unique: false });
    db.close();
  });

  // Tue le gestionnaire « onerror » de l'ouverture → « () => undefined » (la promesse ne finissait jamais)
  it('ouvrirBase rejette si la base existante est d\'une version plus récente', async () => {
    const fabrique = new IDBFactory();
    (await ouvrirSansApplication(fabrique, 'todo-inf563', 2)).close();
    await expect(ouvrirBase({ fabrique })).rejects.toMatchObject({ name: 'VersionError' });
  });

  it('par défaut, utilise l\'IndexedDB du navigateur et l\'heure courante', async () => {
    const fabrique = new IDBFactory();
    vi.stubGlobal('indexedDB', fabrique);
    try {
      const parDefaut = await ouvrirBase();
      await parDefaut.ajouterProjet('Maison');
      const [projet] = await parDefaut.listerProjets();
      parDefaut.fermer();
      expect(projet.creeLe).toMatch(/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{3}Z$/);
      expect(Math.abs(Date.parse(projet.creeLe) - Date.now())).toBeLessThan(60_000);
    } finally {
      vi.unstubAllGlobals();
```

```
    }
  });

  // Tue « fermer: () => db.close() » → « () => undefined »
  it('fermer ferme la connexion : plus aucune lecture n\'est possible', async () => {
    const autre = await ouvrirBase({ fabrique: new IDBFactory() });
    autre.fermer();
    expect(() => autre.listerProjets()).toThrow(expect.objectContaining({ name: 'InvalidStateError' }));
  });
});

describe('T2 · horloge injectée', () => {
  // Tue « maintenant » → « () => undefined » : les dates n'étaient jamais vérifiées.
  // L'horloge est une doublure contrôlée par le test (cours, séance 9 : maîtriser le temps).
  it('date la création, puis la modification sans toucher à la date de création', async () => {
    const creation = '2026-11-11T08:00:00.000Z';
    const modification = '2026-11-11T09:30:00.000Z';
    let maintenant = new Date(creation);
    const datee = await ouvrirBase({ fabrique: new IDBFactory(), horloge: () => maintenant });

    const maison = await datee.ajouterProjet('Maison');
    const id = await datee.ajouterTache({ titre: 'Laver la voiture', projetId: maison });
    maintenant = new Date(modification);
    await datee.modifierTache(id, { terminee: true });

    expect(await datee.listerProjets()).toEqual([{ id: maison, nom: 'Maison', nomNormalise: 'maison', creeLe: creation
}]);
    expect(await datee.listerTaches()).toEqual([
      { id, titre: 'Laver la voiture', projetId: maison, terminee: true, creeLe: creation, modifieLe: modification },
    ]);
    datee.fermer();
  });
});

describe('T2 · suppression en cascade et cas d\'erreur', () => {
  // Tue « curseur.result.continue() » supprimé : la boucle doit être testée avec plusieurs tâches
  it('supprimerProjet supprime toutes les tâches du projet, et seulement elles', async () => {
    const maison = await base.ajouterProjet('Maison');
    const travail = await base.ajouterProjet('Travail');
    await base.ajouterTache({ titre: 'Faire les courses', projetId: maison });
    await base.ajouterTache({ titre: 'Écrire le rapport', projetId: travail });
    await base.ajouterTache({ titre: 'Ranger le garage', projetId: maison });

    await base.supprimerProjet(maison);

    expect((await base.listerProjets()).map((p) => p.nom)).toEqual(['Travail']);
    expect((await base.listerTaches()).map((t) => t.titre)).toEqual(['Écrire le rapport']);
  });

  // Tue « !lecture.result » → « false » et couvre l'annulation (tx.abort, message d'erreur)
  it('modifierTache sur une tâche inexistante rejette et n\'écrit rien', async () => {
    const maison = await base.ajouterProjet('Maison');
    await base.ajouterTache({ titre: 'Faire les courses', projetId: maison });
    const avant = await base.listerTaches();

    await expect(base.modifierTache(999, { titre: 'Fantôme' }))
      .rejects.toMatchObject({ name: 'AbortError', message: 'Transaction annulée' });
    expect(await base.listerTaches()).toEqual(avant);
  });
});
```

Annexe E – Fichiers .feature (inchangés depuis la version 1)

features/01_projets.feature

```
# language: fr
@projets
Fonctionnalité: Organiser mes tâches par projet
  Afin de séparer mes différentes activités
  En tant qu'utilisateur de l'application To-Do
  Je veux créer des projets auxquels mes tâches appartiennent

  Contexte:
    Soit j'ouvre l'application To-Do avec une base vide

  Scénario: Créer un projet
    Quand je crée le projet "Mémoire M2"
    Alors le projet "Mémoire M2" apparaît dans la liste des projets
    Et la base IndexedDB contient exactement les projets :
      | Mémoire M2 |

  Scénario: Refuser un projet sans nom
    Quand je crée le projet "   "
    Alors le message "Le nom du projet est obligatoire." s'affiche
    Et la base IndexedDB ne contient aucun projet

  Plan du scénario: Refuser un projet en double (<cas>)
    Soit le projet "Maison" existe
    Quand je crée le projet "<nom saisi>"
    Alors le message "Le projet « Maison » existe déjà." s'affiche
    Et la base IndexedDB contient exactement les projets :
      | Maison |

    Exemples:
      | cas                 | nom saisi   |
      | même nom            | Maison      |
      | casse différente    | maison      |
      | majuscules          | MAISON      |

  Scénario: Refuser un projet en double saisi avec des espaces autour
    Soit le projet "Maison" existe
    Quand je crée le projet "  Maison  "
    Alors le message "Le projet « Maison » existe déjà." s'affiche
    Et la base IndexedDB contient exactement les projets :
      | Maison |

  Scénario: Filtrer les tâches par projet
    Soit le projet "Maison" existe
    Et le projet "Travail" existe
    Et la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Écrire le rapport" existe dans le projet "Travail"
    Et la tâche "Préparer la soutenance" existe dans le projet "Travail"
    Quand je sélectionne le projet "Travail"
    Alors je vois uniquement les tâches :
      | Écrire le rapport      |
      | Préparer la soutenance |
    Quand je sélectionne tous les projets
    Alors je vois uniquement les tâches :
      | Faire les courses      |
      | Écrire le rapport      |
      | Préparer la soutenance |

  Scénario: Supprimer un projet supprime aussi ses tâches
    Soit le projet "Maison" existe
    Et le projet "Travail" existe
    Et la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Écrire le rapport" existe dans le projet "Travail"
    Quand je supprime le projet "Maison"
    Alors le projet "Maison" n'apparaît plus dans la liste des projets
```

```
    Et je vois uniquement les tâches :
      | Écrire le rapport |
    Et la base IndexedDB contient exactement les tâches :
      | tâche             | projet  | terminée |
      | Écrire le rapport | Travail | non      |
```

features/02_taches.feature

```
# language: fr
@taches
Fonctionnalité: Gérer les tâches d'un projet
  Afin de ne rien oublier
  En tant qu'utilisateur de l'application To-Do
  Je veux ajouter, modifier, terminer et supprimer des tâches rattachées à un projet

  Contexte:
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Maison" existe
    Et le projet "Travail" existe

  Scénario: Ajouter une tâche dans un projet
    Quand j'ajoute la tâche "Payer la facture ENEO" dans le projet "Maison"
    Alors la tâche "Payer la facture ENEO" est affichée dans le projet "Maison"
    Et le compteur indique "1 tâche, dont 0 terminée"
    Et la base IndexedDB contient exactement les tâches :
      | tâche                | projet | terminée |
      | Payer la facture ENEO | Maison | non      |

  Plan du scénario: Refuser une tâche invalide (<cas>)
    Quand j'ajoute la tâche "<titre>" dans le projet "<projet>"
    Alors le message "<message>" s'affiche
    Et la base IndexedDB ne contient aucune tâche

    Exemples:
      | cas               | titre           | projet | message                                |
      | titre vide        |                 | Maison | Le titre de la tâche est obligatoire. |
      | sans projet       | Acheter du pain |        | Choisissez un projet pour la tâche.   |
      | ni titre ni projet |                 |        | Le titre de la tâche est obligatoire. |

  Scénario: Refuser une tâche dont le titre ne contient que des espaces
    Quand j'ajoute la tâche "    " dans le projet "Maison"
    Alors le message "Le titre de la tâche est obligatoire." s'affiche
    Et la base IndexedDB ne contient aucune tâche

  Scénario: Modifier le titre d'une tâche
    Soit la tâche "Apeler le plombier" existe dans le projet "Maison"
    Quand je renomme la tâche "Apeler le plombier" en "Appeler le plombier"
    Alors la tâche "Appeler le plombier" est affichée dans le projet "Maison"
    Et la tâche "Apeler le plombier" n'est plus affichée
    Et la base IndexedDB contient exactement les tâches :
      | tâche               | projet | terminée |
      | Appeler le plombier | Maison | non      |

  Scénario: Déplacer une tâche vers un autre projet
    Soit la tâche "Réviser le chapitre 4" existe dans le projet "Maison"
    Quand je déplace la tâche "Réviser le chapitre 4" vers le projet "Travail"
    Alors la tâche "Réviser le chapitre 4" est affichée dans le projet "Travail"
    Et la base IndexedDB contient exactement les tâches :
      | tâche                | projet  | terminée |
      | Réviser le chapitre 4 | Travail | non      |

  Scénario: Une modification avec un titre vide est refusée
    Soit la tâche "Laver la voiture" existe dans le projet "Maison"
    Quand je renomme la tâche "Laver la voiture" en ""
    Alors le message "Le titre de la tâche est obligatoire." s'affiche
    Et la base IndexedDB contient exactement les tâches :
      | tâche            | projet | terminée |
      | Laver la voiture | Maison | non      |
```

```
  Scénario: Annuler une modification ne change rien
    Soit la tâche "Laver la voiture" existe dans le projet "Maison"
    Quand je commence à renommer la tâche "Laver la voiture" en "Autre chose" puis j'annule
    Alors la tâche "Laver la voiture" est affichée dans le projet "Maison"
    Et la tâche "Autre chose" n'est plus affichée

  Scénario: Marquer une tâche comme terminée puis la réouvrir
    Soit la tâche "Rendre le devoir BDD" existe dans le projet "Travail"
    Quand je coche la tâche "Rendre le devoir BDD"
    Alors la tâche "Rendre le devoir BDD" est marquée comme terminée
    Et le compteur indique "1 tâche, dont 1 terminée"
    Et la base IndexedDB contient exactement les tâches :
      | tâche               | projet  | terminée |
      | Rendre le devoir BDD | Travail | oui      |
    Quand je décoche la tâche "Rendre le devoir BDD"
    Alors la tâche "Rendre le devoir BDD" n'est pas marquée comme terminée

  Scénario: Supprimer une tâche
    Soit la tâche "Faire les courses" existe dans le projet "Maison"
    Et la tâche "Ranger le bureau" existe dans le projet "Maison"
    Quand je supprime la tâche "Faire les courses"
    Alors je vois uniquement les tâches :
      | Ranger le bureau |
    Et la base IndexedDB contient exactement les tâches :
      | tâche            | projet | terminée |
      | Ranger le bureau | Maison | non      |

  Scénario: Un titre contenant du HTML est affiché tel quel
    Quand j'ajoute la tâche "<b>urgent</b> & important" dans le projet "Travail"
    Alors la tâche "<b>urgent</b> & important" est affichée dans le projet "Travail"
```

features/03_persistance_indexeddb.feature

```
# language: fr
@persistance
Fonctionnalité: Sauvegarde des projets et des tâches dans IndexedDB
  Afin de retrouver mon travail après avoir fermé ou rechargé la page
  En tant qu'utilisateur de l'application To-Do
  Je veux que mes projets et mes tâches soient enregistrés dans le navigateur

  Scénario: Une nouvelle base est vide
    Soit j'ouvre l'application To-Do avec une base vide
    Alors la liste des tâches est vide
    Et le compteur indique "Aucune tâche"
    Et la base IndexedDB ne contient aucun projet
    Et la base IndexedDB ne contient aucune tâche

  Scénario: Les données survivent au rechargement de la page
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Maison" existe
    Et la tâche "Arroser les plantes" existe dans le projet "Maison"
    Et je coche la tâche "Arroser les plantes"
    Quand je recharge la page
    Alors la tâche "Arroser les plantes" est affichée dans le projet "Maison"
    Et la tâche "Arroser les plantes" est marquée comme terminée

  Scénario: Les données survivent à la fermeture de l'onglet
    Soit j'ouvre l'application To-Do avec une base vide
    Et le projet "Travail" existe
    Et la tâche "Envoyer le compte rendu" existe dans le projet "Travail"
    Quand je ferme l'onglet et j'ouvre l'application dans un nouvel onglet
    Alors la tâche "Envoyer le compte rendu" est affichée dans le projet "Travail"

  Scénario: L'application affiche les données déjà présentes dans IndexedDB
    Soit la base IndexedDB contient déjà :
      | tâche                  | projet  | terminée |
      | Préparer la réunion    | Travail | non      |
      | Envoyer le compte rendu | Travail | oui      |
      | Réparer le robinet     | Maison  | non      |
```

```
    Quand j'ouvre l'application To-Do
    Alors je vois uniquement les tâches :
      | Préparer la réunion     |
      | Envoyer le compte rendu |
      | Réparer le robinet      |
    Et la tâche "Envoyer le compte rendu" est marquée comme terminée
    Et le compteur indique "3 tâches, dont 1 terminée"
```

Annexe F – Pipeline .github/workflows/ci.yml

.github/workflows/ci.yml

```yaml
# Intégration continue : chaque push (sur n'importe quelle branche) et chaque pull request
# lance les tests unitaires avec couverture, le test de mutation, puis les scénarios BDD.
# Chaque étape a son seuil : sous le seuil, le pipeline est rouge et on ne fusionne pas.
name: ci

on:
  push:
  pull_request:
  workflow_dispatch:   # bouton « Run workflow » pour relancer à la main

jobs:
  tests:
    name: "Tests unitaires, mutation et scénarios BDD (To-Do IndexedDB)"
    # Version figée plutôt que « ubuntu-latest », qui change d'Ubuntu sans prévenir :
    # le pipeline reste reproductible.
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v7

      - uses: actions/setup-node@v7
        with:
          node-version: '24'
          cache: npm

      - name: Installer les dépendances (versions figées par package-lock.json)
        run: npm ci

      - name: Tests unitaires (Vitest) + couverture (Istanbul) - seuils dans vitest.config.js
        run: npm run test:couverture

      - name: Test de mutation (StrykerJS) - seuil dans stryker.config.mjs
        run: npm run test:mutation

      # Pas d'étape « playwright install » (elle restait bloquée sur le téléchargement) :
      # les scénarios utilisent le Google Chrome préinstallé sur la machine.
      - name: Scénarios BDD (Cucumber + Playwright) + couverture - seuils dans .nycrc.json
        run: npm run test:bdd:couverture
        env:
          CANAL: chrome

      - name: Publier les rapports (même en cas d'échec)
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: rapports-tests
          path: rapports/
```

Annexe G – Commandes utiles

Prérequis : Node.js 22 ou plus. Commandes pour PowerShell, depuis le dossier du projet :

```
# Installer les dépendances (versions exactes de package-lock.json)
npm ci
# Navigateur des scénarios : une seule fois, ou bien utiliser Google Chrome avec $env:CANAL="chrome"
npx playwright install chromium

npm run test:unitaires          # 51 tests unitaires
npm run test:couverture         # + couverture Istanbul       → rapports/couverture-unitaire/index.html
npm run test:mutation           # test de mutation StrykerJS  → rapports/mutation/index.html
npm run test:bdd                # les 24 scénarios            → rapports/cucumber/cucumber.html
npm run test:bdd:couverture     # + couverture des scénarios  → rapports/couverture-bdd/index.html
npm run verify                  # les trois mesures, comme la CI

# Démonstration : navigateur visible et ralenti
$env:HEADLESS="false"; $env:SLOWMO="300"; npm run test:bdd
# Une seule fonctionnalité : @projets, @taches ou @persistance
npx cucumber-js --tags "@persistance"
# Utiliser l'application à la main : http://127.0.0.1:8080
npm start
```

Démonstration du test de mutation

Dans `tests/unitaires/regles.test.js`, mettre en commentaire la ligne `[1, '1 terminée'],` et désactiver le test « le compteur est au singulier pour une seule tâche » (`it.skip`) : ce sont les deux seuls tests qui essaient la valeur limite 1. `npm run test:mutation` signale alors de nouveau le survivant `nombre > 1 → nombre >= 1` (`regles.js:64`) et le score tombe à 99,46 %. Rétablir les deux tests : le mutant est de nouveau tué.